PRA1. Processos, pipes i senyals: Suma seqüència d'enters

Alia Ahmad i Arantxa Ballester

PAS 1: Execució concurrent dels processos jugadors i comunicació de sumes parcials utilitzant les crides exit/wait (exit code)

En aquest primer pas, l'objectiu era establir una comunicació bàsica entre el procés pare "controlador" i els processos fills "calculadors" creats amb *fork()*. Per fer-ho, cada calculador retorna la seva suma parcial a través del codi de sortida utilitzant la funció *exit()*. El controlador, per la seva banda, s'espera i recull aquests valors mitjançant la crida *wait()* per anar acumulant la suma total.

No obstant això, tal com analitzem al final del fitxer *calculador_v1.c*, aquest mecanisme falla quan treballem amb seqüències grans. Això és degut a una limitació tècnica de la funció *exit()*: el sistema operatiu només utilitza els 8 bits inferiors per retornar el valor. Conseqüentment, qualsevol suma que superi 255 pateix un desbordament (*overflow*), generant un resultat erroni i fent necessari l'ús de canonades en els passos posteriors

PAS 2: Comunicació de sumes parcials utilitzant pipes:

A diferència del Pas 1 (on la suma parcial s'enviava mitjançant el codi de sortida de 'exit'), al Pas 2 les sumes parcials es transmeten del calculador al controlador mitjançant la canonada 'pipeSumes'.

S'utilitza 'dup2' per a la redirecció de descriptors de fitxer:

- Calculador (fill): Redirigeix l'extrem d'escriptura de la pipe al descriptor de fitxer 20.
- Controlador (pare): Redirigeix l'extrem de lectura de la pipe al descriptor de fitxer 21.

Tant el pare com els fills han de tancar els descriptors originals de la canonada per evitar fuites de descriptors i permetre que el protocol de tancament de pipes funcioni correctament. I enviar un resultat com una variable 'long long' per la pipe permet transmetre valors molt més grans sense tenir la restricció de 8 bits, que hi havia a l''exit code' del Pas 1.

PAS 3: Treball amb més d'una seqüència

El programa "evoluciona" i passa a ser un programa interactiu que sol·licitar per teclat i processa diverses seqüències de forma interactiva al bucle fins que l'usuari decideix sortir. Els *N* calculadors es creen una sola vegada a l'inici del programa mitjançant 'fork() + execl()'. No moren després d'un càlcul, sinó que es queden en un bucle 'while (read(...) > 0)'.

S'hi afegeix una segona canonada ('pipeRangs') per enviar els subrangs des del controlador cap als calculadors. I es redirigeix la seva escriptura al *descriptor 10* al controlador i la lectura *al descriptor 11* als calculadors.

Quan l'usuari indica que no vol més seqüències, el controlador executa 'close(10)'. Els calculadors llegeixen 0 bytes al descriptor 11, detecten *EOF* (Fin de Fitxer), surten del bucle while i finalitzen amb 'exit(0)'.

PAS 4: Tractament del senyal SIGINT (Ctrl+C):

En aquest últim pas, hem afegit el tractament de senyals per garantir que el sistema sigui robust i respongui correctament a les interrupcions. Concretament, quan l'usuari prem *Ctrl+C*, el "controlador" rep el senyal *SIGINT* i s'encarrega d'enviar un senyal *SIGTERM* als processos "calculador" perquè aturin els càlculs i finalitza la seva execució de manera neta.

Per aconseguir-ho, hem configurat el "calculador" perquè ignori el *SIGINT* i reaccioni exclusivament al *SIGTERM*. A més, per complir amb les restriccions tècniques de l'assignatura, hem evitat l'ús de funcions no segures com *printf* dins del gestor de senyals, substituint-les per la crida al sistema *write* per mostrar els missatges informatius a la sortida estàndard.